

Algorithms & Complexity — Խորը ուղեցույց

Big O, Binary Search, Vector/Map/Hash complexity

Այս ուղեցույցը մանրամասն բացատրում է algorithm complexity-ի հիմնական Big O notation, binary search, vector find, map insertion, hash table-ի ashxatangy, orinaknerov ev interview harcerov.

1. Ի՞նչ է Big O Notation-ը

Big O-ն նկարագրում է algorithm-ի **aracutyany/hishoxutyany achy** kaxvac input-ի chapic (n), erb n-y mecanum e. Cuyc e talis **worst-case** verin sahmany.

```
0(1)          -> hastatun (input-ic ankax)
0(log n)      -> logarithmic (binary search)
0(n)          -> linear (mek loop)
0(n log n)    -> merge/quick sort, efficient sort
0(n^2)        -> knestic loop (bubble sort)
0(2^n)        -> exponential (naive recursion)
0(n!)         -> factorial (permutations)
```

Achi karg (arag -> dandaghn).

```
0(1) < 0(log n) < 0(n) < 0(n log n) < 0(n^2) < 0(2^n) < 0(n!)
```

2. Big O orinakner

O(1) — hastatun

```
int x = arr[5];           // unix mutq, input-ic ankax
v.push_back(x);           // amortized O(1)
```

$O(n)$ — linear

```
for (int i = 0; i < n; i++)    // n angam
    cout << arr[i];
```

$O(n^2)$ — knestic

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)    // n * n
        cout << arr[i][j];
```

$O(\log n)$ — halved amen qayl

```
while (n > 1)
    n = n / 2;    // amen qayl kisum -> log n
```

3. Binary Search Complexity

Binary search-y **sorted** array-um voronum e, amen qayl **kislov** voronman tiroyty.

```
int binarySearch(vector<int>& v, int target) {
    int l = 0, r = v.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (v[mid] == target) return mid;
        else if (v[mid] < target) l = mid + 1;    // aj kes
        else r = mid - 1;                        // dzax kes
    }
    return -1;
}
```

Amen qayl tiroyty kisum -> $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$
Qayleri qanak = $\log_2(n)$

Complexity. $O(\log n)$ (space $O(1)$). Payman. array-y petq e **sorted** lini.

```
n = 1000000 -> ~20 qayl (log2(1000000))  
Linear search-y kliner 1000000 qayl!
```

4. Vector-nıŭ element qtnely

find (unsorted) — $O(n)$

```
auto it = find(v.begin(), v.end(), target); //  $O(n)$  - amen element stugum
```

binary_search (sorted) — $O(\log n)$

```
sort(v.begin(), v.end()); //  $O(n \log n)$  mek angam  
binary_search(v.begin(), v.end(), x); //  $O(\log n)$ 
```

index-ov access — $O(1)$

```
int x = v[5]; //  $O(1)$  - uxix
```

Vector search:

unsorted find	-> $O(n)$
sorted binary	-> $O(\log n)$
index access	-> $O(1)$

5. Map-nıŭ insertion

std::map (Red-Black Tree)

```
map<int, string> m;  
m[5] = "hello"; //  $O(\log n)$  - tree-i mej tex gtnel + balance  
m.insert({3, "x"}); //  $O(\log n)$ 
```

$O(\log n)$ -- balanced tree-i mej tex gtnely logarithmic e.

std::unordered_map (Hash Table)

```
unordered_map<int, string> m;  
m[5] = "hello";           // O(1) average, O(n) worst
```

O(1) average (hash), **O(n) worst** (collision/rehash)。

```
map          insert -> O(log n)      (sorted, RB-tree)  
unordered_map insert -> O(1) average (hash)
```

6. Hash Table ինչպես է աշխատում

Hash table-y key-y verchum e **hash function**-i mijov -> stanum **bucket index** -> element pahvum ayd bucket-um.

```
key --> hash(key) --> index --> bucket[index]
```

```
hash("apple") = 3 -> bucket[3]  
hash("banana") = 7 -> bucket[7]
```

Lookup

1. hash(key) hashvel -> O(1)
2. bucket gtnel -> O(1)
3. bucket-um element gtnel -> O(1) average (qich collision)

Collision

Erb erku key nuyn bucket- n en stanum.

```
hash("cat") = 3  
hash("dog") = 3      <- collision  
bucket[3]: (cat) -> (dog) [chaining - linked list]
```

Shat collision -> bucket-i list erkarum -> lookup O(n) darnum.

Load factor & rehash

load factor = element-ner / bucket-ner

Gerazancelis (~1) -> rehash: bucket-ner x2, bolor element verabashxum ($O(n)$)

7. Complexity Cheat (STL container-ner)

Container	Access	Search	Insert	Delete
vector	$O(1)$	$O(n)$	$O(1)^*$ end	$O(n)$ mid
list	$O(n)$	$O(n)$	$O(1)$	$O(1)$
map/set	-	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_map/set	-	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
stack/queue	-	-	$O(1)$	$O(1)$
priority_queue	$O(1)$ top	-	$O(\log n)$	$O(\log n)$

* amortized.

8. Interview harcer & patasxanner

Big O inch e?

Algorithm-i aracutyany/hishoxutyany achy input-i chapy (n); worst-case verin sahman.

Binary search complexity?

$O(\log n)$ - amen qayl tiroyty kisum. Payman: sorted.

Vector-um element gtnel?

unsorted find $O(n)$; sorted binary_search $O(\log n)$; index access $O(1)$.

Map insert complexity?

std::map $O(\log n)$ (RB-tree); unordered_map $O(1)$ average (hash).

Hash table inhpes?

hash(key) -> bucket index -> pahel/gtnel. $O(1)$ average, $O(n)$ worst (collision).

Inchu binary search $O(\log n)$?

Amen qayl input-y kisum ($n \rightarrow n/2 \rightarrow \dots \rightarrow 1$) -> $\log_2(n)$ qayl.

$O(n \log n)$ inch algoritm?

Efficient sort (merge, quick, heap).

map vs unordered_map complexity?

map $O(\log n)$ sorted; unordered_map $O(1)$ avg unsorted.

Cheat Sheet

BIG O (arag -> dandaghn):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

BINARY SEARCH: $O(\log n)$, sorted array, amen qayl kisum

$n=1M \rightarrow \sim 20$ qayl

VECTOR SEARCH: unsorted $O(n)$ | sorted binary $O(\log n)$ | index $O(1)$

MAP INSERT: map $O(\log n)$ (RB-tree) | unordered_map $O(1)$ avg (hash)

HASH TABLE: hash(key) -> bucket -> $O(1)$ avg, $O(n)$ worst (collision)

load factor $> \sim 1 \rightarrow$ rehash (x2 buckets, $O(n)$)

STL: vector access $O(1)$ | map $O(\log n)$ | unordered $O(1)$ avg | heap top $O(1)$